

category: avalonia

tags: [migration, winforms, avalonia, mvvm, binding]

severity: high

created: 2026-06-24

updated: 2026-06-24

project-origin: iPGSv4 (branch zcu-avalonia)

Avalonia Migration từ WinForms: Pitfall và Pattern Bắt Buộc

Tình huống gặp phải

Chuyển đổi toàn bộ UI từ Windows Forms sang Avalonia UI (cross-platform) cho dự án iPGSv4. Kiến trúc cũ dùng WinForms controls trực tiếp, code-behind nặng, không có MVVM.

Các pitfall phổ biến khi migrate

1. Thread / Dispatcher

WinForms:

```
this.Invoke(() => label.Text = value);
```

Avalonia (ĐÚNG):

```
Dispatcher.UIThread.Post(() => label.Content = value);  
// hoặc dùng async  
await Dispatcher.UIThread.InvokeAsync(() => label.Content = value);
```

⚠ Không dùng `Control.Invoke` — không tồn tại trong Avalonia.

2. Property binding — PHẢI implement INotifyPropertyChanged

WinForms cho phép set trực tiếp `control.Text = x`. Avalonia binding không hoạt động nếu ViewModel không implement INPC:

```
// ViewModel bắt buộc:  
public class MyViewModel : ReactiveObject // hoặc ObservableObject  
{  
    private string _name;  
    public string Name  
    {  
        get => _name;  
        set => this.RaiseAndSetIfChanged(ref _name, value); // ReactiveUI  
    }  
}
```

3. MessageBox → không có trong Avalonia mặc định

WinForms: `MessageBox.Show("msg")`

Avalonia — dùng dialog service hoặc:

```
// Dùng thư viện MsBox.Avalonia:
var box = MessageBoxManager.GetMessageBoxStandard("Title", "Message");
await box.ShowDialog();
```

Hoặc tự tạo dialog window qua `ShowDialog<T>()`.

4. Controls đổi tên / namespace

WinForms	Avalonia
---	---
Label	TextBlock (display) hoặc Label
TextBox	TextBox (tương tự)
ComboBox	ComboBox (binding khác)
PictureBox	Image
Panel	Panel / StackPanel / Grid
GroupBox	GroupBox
DataGridView	DataGrid
Form	Window
UserControl	UserControl

5. Navigation pattern trong Avalonia (thay vì Form.Show())

WinForms:

```
var form = new CameraForm();
form.Show();
```

Avalonia — dùng ContentControl + ViewModel routing:

```
<ContentControl Content="{Binding CurrentView}" />
// Trong MainViewModel:
public ViewModelBase CurrentView
{
    get => _currentView;
    set => this.RaiseAndSetIfChanged(ref _currentView, value);
}

// Navigate:
CurrentView = new CameraViewModel();
```

Đăng ký DataTemplate trong `App.axaml`:

```
<Application.DataTemplates>
  <DataTemplate DataType="vm:CameraViewModel">
    <views:CameraView />
  </DataTemplate>
</Application.DataTemplates>
```

6. Event handling — code-behind vẫn OK nhưng hạn chế

Avalonia hỗ trợ code-behind nhưng khuyến khích Command binding:

```
<!-- Thay vì Button.Click event -->
<Button Command="{Binding SaveCommand}" Content="Save" />

public ICommand SaveCommand { get; }
// Khởi tạo:
SaveCommand = ReactiveCommand.Create(ExecuteSave);
```

Áp dụng lại

- Khi thấy `this.Invoke` trong code cũ → thay bằng `Dispatcher.UIThread.Post`
- Khi binding không cập nhật UI → kiểm tra ViewModel có INPC không
- Khi cần navigate → dùng ContentControl + ViewModel routing, KHÔNG tạo Window mới tùy tiện
- Trước khi dùng control WinForms nào → tra bảng đổi tên phía trên

Chú ý / Cạm bẫy

- ⚠ `DataContext` trong Avalonia hoạt động khác WinForms — phải set đúng trong constructor hoặc XAML
- ⚠ Avalonia dùng `.axaml` không phải `.xaml` (dù nội dung giống nhau)
- ⚠ `async void` event handler cần `try/catch` toàn bộ — exception trong `async void` sẽ crash app
- ⚠ ComboBox binding cần `ItemsSource` + `SelectedItem` — không dùng `SelectedValue` như WPF

Tham chiếu

- Docs: <https://docs.avaloniaui.net/docs/get-started/migrate-from-wpf> (gần giống WPF)
- Project: iPGSv4, branch `zcu-avalonia`
- ReactiveUI: <https://www.reactiveui.net/docs/>